

Data Transformation DSL

This DSL is a **domain-specific language for transforming lists of data in a clear, declarative, and procedural way**. It allows you to define a list, either by reading input from a user (**input fruits**) or by assigning a static value (**list numbers = [1, 2, 3]**), and then apply a sequence of transformations and operations on it. These transformations include adding or removing elements, concatenating or multiplying lists, mapping each element with an expression (**map numbers x => x + 1**), filtering elements based on conditions, sorting, joining into a string, removing duplicates, and producing output. The language is designed to work on **lists only**, keeping the syntax simple and predictable.

The DSL also supports **control flow constructs** like **if** and **for**, allowing you to conditionally execute operations or iterate over list elements. For example, you can write **if size(numbers) > 5 ... end** to check the size of a list and perform operations only if the condition holds, or you can loop through each element of a list using **for x in numbers ... end**, optionally combining it with nested **if** statements inside the loop. These constructs give you expressive power to not only transform lists but also react to their state or contents, enabling more dynamic and flexible scripts.

Finally, the DSL is designed to be **easy to read and write**, with one operation per line and minimal syntax overhead. It avoids nesting, uses clear verbs (**add, map, filter, sort, output**), and allows arithmetic and logical expressions in a concise way (**x => x * 10, x => length(x) > 5**). It strikes a balance between being declarative—where you describe what you want done—and imperative—where you control the order of operations. Overall, it provides a simple, scriptable way to preprocess and manipulate lists interactively or programmatically.